

实 验 报 告

实验名称: 实验三 Hebb 学习

课程名称: 认知科学与类脑计算

实验地点: K2 227

实验日期: 2025/10/10

班 级: 23 级智能班

姓 名: 宋浩宇

学 号: 202300130183

一 实验目的：

加深对 Hebb 学习模型的理解，能够使用 Hebb 学习模型解决简单问题

二 实验环境：

主系统：Windows 11 家庭中文版 23H2 ; ArchLinux

编辑器：Visual Studio Code ; PyCharm 2025.1.2 ; neovim 0.11

Python 解释器：Python 3.9

Pytorch 版本：2.8.0

CUDA 版本：12.8.97

硬件环境：

CPU：13th Gen Intel(R) Core(TM) i9-13980HX 2.20 GHz

内存：32.0 GB (31.6 GB 可用)

磁盘驱动器：NVMe WD_BLACKSN850X2000GB

显示适配器：NVIDIA GeForce RTX 4080 Laptop GPU

CUDA 核心数：7424

三 实验内容：

根据 Hebb 学习模型的相关知识，使用 Python 语言实现一个简单 Hebb 学习模型。

四 实验步骤：

1. 实验程序：

```
# 数字点阵
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors
cmap = mcolors.ListedColormap(['white', 'black'])
bounds = [-1.5, 0, 1.5]
norm = mcolors.BoundaryNorm(bounds, cmap.N)
one = np.array([[ -1, -1, 1, -1, -1],
                 [-1, 1, 1, -1, -1],
                 [-1, -1, 1, -1, -1],
                 [-1, -1, 1, -1, -1],
                 [-1, -1, 1, -1, -1],
                 [-1, 1, 1, 1, -1]]).flatten()
two = np.array([[ 1, 1, 1, 1, 1],
                 [-1, -1, -1, -1, 1],
                 [-1, -1, -1, -1, 1],
                 [ 1, 1, 1, 1, 1],
                 [ 1, -1, -1, -1, -1],
                 [ 1, 1, 1, 1, 1]]).flatten()
zero = np.array([[ 1, 1, 1, 1, 1],
```

```

        [1, -1, -1, -1, 1],
        [1, -1, -1, -1, 1],
        [1, -1, -1, -1, 1],
        [1, -1, -1, -1, 1],
        [1, 1, 1, 1, 1]]).flatten()
def one_dim_array_to_two_dim(array):
    array = array.reshape(6, 5)
    return array
def save_number_image(number, label, path):
    plt.figure(figsize=(3, 5))
    plt.imshow(number, cmap=cmap, norm=norm)
    plt.title(label)
    plt.axis('off')
    plt.savefig(path)
    plt.close()
def softmax(x, axis=-1):
    x_max = np.max(x, axis=axis, keepdims=True) # 防溢出
    exp = np.exp(x - x_max)
    return exp / np.sum(exp, axis=axis, keepdims=True)
# 定义 Hebb 神经网络
class HebbNet:
    def __init__(self, learning_rate=0.1, epochs=1000,
train_data_set=None, train_label_set=None):
        self.learning_rate = learning_rate
        self.epochs = epochs
        self.train_data_set = train_data_set
        self.train_label_set = train_label_set
        if train_data_set is not None and train_label_set is not None:
            self.weights = np.zeros((len(train_label_set[0]),
len(train_data_set[0])))
            # print("初始化权重为:", self.weights)
    def train_no_oja(self):
        for epoch in range(self.epochs):
            for i in range(len(self.train_data_set)):
                self.weights += self.learning_rate *
np.outer(self.train_label_set[i], self.train_data_set[i])
            # print("第", epoch, "轮权重为:", self.weights)
    def train_yes_oja(self):
        for epoch in range(self.epochs):
            for i in range(len(self.train_data_set)):
                x = self.train_data_set[i]
                t = self.train_label_set[i]
                for j in range(len(t)):
                    if t[j] != 0:

```

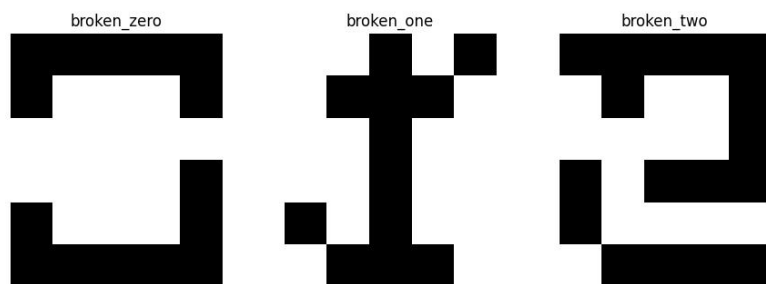
```

        y_post = t[j] # 目标作为后突触活动, 通常为 1
        activation = np.dot(self.weights[j], x) #
前向计算激活
        self.weights[j] += self.learning_rate *
y_post * (x - activation * self.weights[j])
    def predict(self, input_data):
        return softmax(np.matmul(self.weights, input_data))
def random_broke_number(number, broke_count=-1):
    if broke_count == -1:
        broke_count = np.random.randint(0, len(number) / 2)
    for i in range(broke_count):
        index = np.random.randint(0, len(number))
        number[index] = -1 if number[index] == 1 else 1
    return number
if __name__ == '__main__':
    train_data_set = [one, two, zero]
    train_label_set = [[0,1,0], [0,0,1], [1,0,0]]
    hebb_net = HebbNet(train_data_set=train_data_set,
train_label_set=train_label_set, epochs=100)
    hebb_net.train_yes_oja()
    print(hebb_net.weights)
    broken_one = random_broke_number(one, 3).flatten()
    broken_two = random_broke_number(two, 3).flatten()
    broken_zero = random_broke_number(zero, 3).flatten()
    save_number_image(one_dim_array_to_two_dim(broken_one),
'broken_one', 'broken_1.png')
    save_number_image(one_dim_array_to_two_dim(broken_two),
'broken_two', 'broken_2.png')
    save_number_image(one_dim_array_to_two_dim(broken_zero),
'broken_zero', 'broken_0.png')
    print(hebb_net.predict(broken_zero))
    print(hebb_net.predict(broken_one))
    print(hebb_net.predict(broken_two))

```

2. 程序实验结果:

首先用于预测的加入了噪声的数字长这样:



然后权重和预测结果是这样：

```
[[ 0.18257419  0.18257419  0.18257419  0.18257419  0.18257419  0.18257419
 -0.18257419 -0.18257419 -0.18257419  0.18257419  0.18257419 -0.18257419
 -0.18257419 -0.18257419  0.18257419  0.18257419 -0.18257419 -0.18257419
 -0.18257419  0.18257419  0.18257419 -0.18257419 -0.18257419 -0.18257419
  0.18257419  0.18257419  0.18257419  0.18257419  0.18257419  0.18257419]
 [-0.18257419 -0.18257419  0.18257419 -0.18257419 -0.18257419 -0.18257419
  0.18257419  0.18257419 -0.18257419 -0.18257419 -0.18257419 -0.18257419
  0.18257419 -0.18257419 -0.18257419 -0.18257419 -0.18257419  0.18257419
 -0.18257419 -0.18257419 -0.18257419 -0.18257419  0.18257419 -0.18257419
 -0.18257419 -0.18257419  0.18257419  0.18257419  0.18257419 -0.18257419]
 [ 0.18257419  0.18257419  0.18257419  0.18257419  0.18257419 -0.18257419
 -0.18257419 -0.18257419 -0.18257419  0.18257419 -0.18257419 -0.18257419
 -0.18257419 -0.18257419  0.18257419  0.18257419  0.18257419  0.18257419
  0.18257419  0.18257419  0.18257419 -0.18257419 -0.18257419 -0.18257419
 -0.18257419  0.18257419  0.18257419  0.18257419  0.18257419  0.18257419]]
```

以上为最后的权重，

```
[0.80594673 0.00699425 0.18705902]
```

```
[0.00412783 0.98730404 0.00856813]
```

```
[0.18566215 0.01440953 0.79992832]
```

以上为预测的概率，从左到右分别是 0，1，2 的概率。

3. 分析和改进：

从结果上来看，网络预测的是相当准确的。改进的话，已经融入到我的代码里了。从代码里可以看到，最初我是没有使用 oja 规则的，在最初的实验中，这会导致权重爆炸，权重的值会随着 epoch 增加而变得非常非常大，而当我们引入 oja 规则的改进后，权重变得集中且收敛了。

五 实验小结：

从结果来看，对于这三个数字的预测，网络的预测效果还是比较不错的，而且加入的噪声即使比较多也不会有问题，接下来可以考虑试试把 10 个数字都加入网络来测试效果了。